```
0000 separate(¬IDL.SEM_PHASE )¶--|-------------------------------------------------------------------------------------------¶--|¬¬PROCEDURE¬FIX_PRE¶
     --|-------------------------------------------------------------------------------------------¶procedure¬FIX_PRE is¶   ¶  --|-------------------
     ------------------------------------------------------------------------------¶  --|¬¬FUNCTION COPY_NODE¶  function COPY_NODE ( NODE :¬TREE ) return TRE
     E is¶  begin¶    if NODE.PT = HI¬or NODE.PT = S then¬return NODE;¬¬¬¬¬--| PAS DE COPIE DE¬BLOC ATTRIBUTS DANS¬CES CAS¶    else¶      declare¶      L
     EN¬: ATTR_NBR¬:= DABS( 0, NODE ).NSIZ;¶        RESULT¬: TREE¬:= MAKE( NODE.TY, LEN );¶      begin¶        for I in 0 .. LEN loop¶¬DABS( I, RESULT, DA
     BS( I, NODE ) );¶       end loop;¶       return RESULT;¶     end;¶   end if;¶  end COPY_NODE;¶  --|-----------------------------------------
     ------------------------------------------¶  --|¬¬PROCEDURE¬ABORT_RUN¶  procedure ABORT_RUN ( MSG :STRING ) is¶  begin¶    SET_OUTPUT( STANDARD_
     OUTPUT );¶    PUT( "**** " );¶    PUT_LINE( MSG );¶  end ABORT_RUN;¶  --|-------------------------------------------------------------------
     ----------¶  --|¬PROCEDURE¬DEFINE_ID¶   procedure DEFINE_ID ( ID :TREE ) is¶   begin¶     if ID.TY not in CLASS_ENUM_LITERAL¬and then ID.TY /=
      DN_CONSTANT_ID then¶       PUT_LINE( PRINT_NAME( D( LX_SYMREP, ID ) )¬);¶     end if;¶    LIST( D( LX_SYMREP, ID ), SINGLETON( ID ) );¬¬¬¬¬¬--| METT
0010 RE LE SINGLETON ID DANS LA XD_DEFLIST DU SYMBOLE (DOUBLE CHANAGE ENTRE¬LES DEUX)¶   end DEFINE_ID;¶   --|-------------------------------------
     ------------------------------------------¶   --|¬PROCEDURE¬HEAD_DEFN¶   function HEAD_DEFN ( USED_ID_OR_SYMREP :TREE )¬return TREE is¶     SYM
     REP¬: TREE¬:= USED_ID_OR_SYMREP;¶     DEFLIST¬: SEQ_TYPE;¶   begin¶     if USED_ID_OR_SYMREP.TY /= DN_SYMBOL_REP then¬¬¬¬¬¬--| UN USED_NAME_ID¬OU USED_
     OBJECT_ID¶       SYMREP := D(¬LX_SYMREP, USED_ID_OR_SYMREP );¶     end if;¶     DEFLIST := LIST( SYMREP );¶       ¶     if IS_EMPTY( DEFLIST ) then¶
         PUT( "SYMBOL NOT DEFINED -- " );¶       PUT_LINE( PRINT_NAME( SYMREP ) );¶       return TREE_VOID;¶     else¶       return HEAD(¬DEFLIST );¶     e
     nd if;¶  end HEAD_DEFN;¶    --|-------------------------------------------------------------------------------------------¶    --|¬PROCEDURE¬GET_BAS
     E_TYPE¶   function GET_BASE_TYPE ( TYPE_SPEC :TREE ) return TREE is¶    begin¶      if TYPE_SPEC.TY in CLASS_NON_TASK¬then¶       return D( SM_BASE_
     TYPE, TYPE_SPEC );¶      else¶       return TYPE_SPEC;¶      end¬if;¶    end GET_BASE_TYPE;¶    --|-----------------------------------------
     ------------------------------------------¶    --|¬PROCEDURE¬TYPE_SPEC_FOR_SUBTYPE¶    function TYPE_SPEC_FOR_SUBTYPE ( SUBTYPE_INDICATION :TREE ) re
     turn TREE is¶      CONSTRAINT¬¬: TREE;¶      NAME¬¬: TREE;¶      BASE_TYPE¬¬: TREE;¶      use¬MAKE_NOD;¶    begin¶      if SUBTYPE_INDICATION.TY = DN_
0020 SUBTYPE_INDICATION then¶       CONSTRAINT := D( AS_CONSTRAINT,¬SUBTYPE_INDICATION );¶       NAME := D( AS_NAME, SUBTYPE_INDICATION );¶      else¶
          CONSTRAINT := TREE_VOID;¶       NAME := SUBTYPE_INDICATION;¶      end¬if;¶      ¶      BASE_TYPE := D( SM_TYPE_SPEC, D( SM_DEFN, NAME ) );¶
      if CONSTRAINT¬= TREE_VOID then¶       return BASE_TYPE;¶     elsif BASE_TYPE.TY = DN_INTEGER then¶       D( SM_TYPE_SPEC, CONSTRAINT, BASE_TYPE);¶
          D( SM_EXP_TYPE, D ( AS_EXP1, CONSTRAINT),¬BASE_TYPE¬);¶       D( SM_EXP_TYPE, D ( AS_EXP2, CONSTRAINT),¬BASE_TYPE¬);¶       return MAKE_INTE
     GER (¶¬¬SM_RANGE => CONSTRAINT,¶¬¬SM_BASE_TYPE => BASE_TYPE,¶¬¬XD_SOURCE_NAME => D( XD_SOURCE_NAME, BASE_TYPE ) );¶      end¬if;¶      ABORT_RUN ( "BA
     D TYPE FOR SUBTYPE_INDICATION" );¶      raise PROGRAM_ERROR;¶    end TYPE_SPEC_FOR_SUBTYPE;¶    --|-------------------------------------
     ------------------------------------------¶    --|¬PROCEDURE¬WALK¶    procedure WALK ( NODE, PARENT, REGION :TREE )¬is¶      use¬MAKE_NOD;¶    begin¶
          case NODE.TY is¶       when DN_VOID =>¶       null;¶       when DN_CONSTANT_ID =>¶       DEFINE_ID( NODE );¶       D( XD_REG
     ION, NODE, REGION );¶       when DN_ENUMERATION_ID | DN_CHARACTER_ID =>¶       DEFINE_ID( NODE );¶       D( XD_REGION, NODE, REGION );¶
          ¶     when DN_TYPE_ID =>¶       DEFINE_ID( NODE );¶       D( SM_FIRST, NODE, NODE);¶       D( XD_REGION, NODE, REGION );¶       ¶     when
0030  DN_SUBTYPE_ID =>¶       DEFINE_ID( NODE );¶       D( XD_REGION, NODE, REGION );¶       ¶     when DN_PACKAGE_ID =>¶       DEFINE_ID( NODE );¶
          D( SM_FIRST, NODE, NODE));¶       D( SM_SPEC,¬NODE, D (¬AS_HEADER, PARENT )¬);¶       D( SM_UNIT_DESC, NODE, TREE_VOID );¶       D( XD_REGION,
      NODE, REGION );¶       ¶     when DN_EXCEPTION_ID =>¶       DEFINE_ID( NODE );¶       D( SM_RENAMES_EXC, NODE, TREE_VOID );¶       D( XD_REGION
     , NODE, REGION );¶       ¶     when DN_CONSTANT_DECL =>¶       declare¶¬SOURCE_NAME_S¬: constant TREE := D( AS_SOURCE_NAME_S,¬NODE );¶¬EXP¬: cons
     tant TREE := D( AS_EXP,¬NODE );¶¬TYPE_DEF¬: constant TREE := D( AS_TYPE_DEF, NODE¬);¶       begin¶¬WALK ( SOURCE_NAME_S, NODE, REGION );¶¬WALK ( EXP
     , NODE, REGION );¶¬WALK ( TYPE_DEF, NODE, REGION¬);¶¬declare¶¬  HEAD_SOURCE_NAME¬: TREE¬:= HEAD( LIST( D( AS_SOURCE_NAME_S, NODE ) ) );¶¬begin¶¬  D( S
     M_OBJ_TYPE, HEAD_SOURCE_NAME, TYPE_SPEC_FOR_SUBTYPE( D( AS_TYPE_DEF, NODE¬) ) );¶¬  D( SM_INIT_EXP, HEAD_SOURCE_NAME, D( AS_EXP, NODE ) );¶¬end;¶
     end;¶       ¶     when DN_EXCEPTION_DECL =>¶       declare¶¬SOURCE_NAME_S¬: constant TREE¬:= D( AS_SOURCE_NAME_S, NODE );¶       begin¶¬WALK(
     SOURCE_NAME_S, NODE, REGION );¶       end;¶       ¶     when DN_TYPE_DECL =>¶       declare¶¬SOURCE_NAME¬: constant TREE¬:= D( AS_SOURCE_NAME, NO
     DE );¶¬DSCRMT_DECL_S¬: constant TREE¬:= D( AS_DSCRMT_DECL_S, NODE );¶¬TYPE_DEF¬¬: constant TREE¬:= D( AS_TYPE_DEF, NODE );¶       begin¶¬WALK( SOURCE
0040 _NAME, NODE, REGION );¶¬WALK( DSCRMT_DECL_S, NODE, REGION );¶¬WALK( TYPE_DEF, NODE, REGION );¶     ¶¬¬¬¬¬-- SAVE ANCESTOR TYPE NAME FOR DERIVED TYPE (
     _ADDRESS)¶¬if TYPE_DEF.TY = DN_DERIVED_DEF then¶¬  D( SM_SPEC, SOURCE_NAME, D( SM_DEFN, D( AS_NAME, D (¬AS_SUBTYPE_INDICATION, TYPE_DEF) ) ) );¶¬
     end if;¶       end;¶       ¶     when DN_SUBTYPE_DECL =>¶       declare¶¬SOURCE_NAME¬: constant TREE := D( AS_SOURCE_NAME, NODE );¶¬SUBTYPE_INDIC
     ATION¬: constant TREE := D( AS_SUBTYPE_INDICATION, NODE¬);¶¬SUBTYPE_NODE¬: TREE;¶       begin¶¬WALK( SOURCE_NAME, NODE, REGION );¶¬WALK( SUBTYPE_INDI
     CATION, NODE, REGION );¶¬     ¶¬SUBTYPE_NODE := TYPE_SPEC_FOR_SUBTYPE( SUBTYPE_INDICATION );¶¬D( SM_TYPE_SPEC, D(¬AS_SOURCE_NAME, NODE ), SUBTYPE_NODE
      );¶¬if D( AS_CONSTRAINT, SUBTYPE_INDICATION¬) /= TREE_VOID then¶¬  D( XD_SOURCE_NAME, SUBTYPE_NODE, D(  AS_SOURCE_NAME, NODE¬) );¶¬end if;¶       en
     d;¶       ¶     when DN_PACKAGE_DECL =>¶     declare¶¬       SOURCE_NAME¬: constant TREE := D ( AS_SOURCE_NAME, NODE );¶       HEADER¬: constant T
     REE := D ( AS_HEADER, NODE );¶       UNIT_KIND¬: constant TREE := D ( AS_UNIT_KIND, NODE );¶     begin¶       WALK ( SOURCE_NAME, NODE, REGION );
     ¶       WALK ( HEADER, NODE, REGION => SOURCE_NAME );¶       WALK ( UNIT_KIND, NODE, REGION => SOURCE_NAME );¶     end;¶       ¶     when DN_ENUMER
     TH_ENUM_REP =>¶     declare¶       NAME¬: constant TREE¬:= D ( AS_NAME, NODE );¶       EXP¬: constant TREE¬:= D ( AS_EXP, NODE¬);¶     begin¶¬¬¬

0050 ¬¬-ONLY WALK PREFIX OF ATTRIBUTE, NOT ATTRIBUTE NAME¶¬       WALK ( D ( AS_NAME, NAME), NODE, REGION );¶¬       WALK ( EXP,¬NODE, REGION );¶¬¬¬¬¬-- I
     N SOURCE TO INDICATE CD_IMPL_SIZE FOR A TYPE¶¬¬¬¬¬-- FORM IS: FOR NNN'SIZE USE 999;¶¬       D¬( CD_IMPL_SIZE, D (¬SM_TYPE_SPEC, D ( SM_DEFN, D ( AS_NA
     ME,¬NAME))), D ( SM_VALUE, EXP ) );¶¬     end;¶       ¶  when DN_PRAGMA =>¶¬     declare¶¬       USED_NAME_ID¬: constant TREE := D ( AS_USED_NAME_
     ID,¬NODE );¶¬       GENERAL_ASSOC_S¬: constant TREE := D ( AS_GENERAL_ASSOC_S, NODE );¶¬     begin¶¬       WALK ( USED_NAME_ID, NODE, REGION);¶¬
         WALK ( GENERAL_ASSOC_S, NODE, REGION);¶¬¬¬¬¬-- IN SOURCE TO INDICATE PRAGMA PACKED (STRING)¶¬       DB ( SM_IS_PACKED, D ( SM_TYPE_SPEC, D ( SM_DE
     FN, HEAD ( LIST¬( GENERAL_ASSOC_S )¬) ) ), TRUE );¶¬     end;¶       ¶  when DN_ENUMERATION_DEF =>¶¬     declare¶¬       ENUM_LITERAL_S¬: constant
      TREE¬:= D ( AS_ENUM_LITERAL_S, NODE);¶¬       RANGE_NODE¬: TREE¬:= MAKE (¬DN_RANGE );¶¬       ENUMERATION¬: TREE¬:= MAKE_ENUMERATION¬(¶¬¬¬¬SM_LITER
     AL_S¬=> ENUM_LITERAL_S,¶¬¬¬¬SM_RANGE¬=> RANGE_NODE,¶¬¬¬¬CD_IMPL_SIZE¬=> 8,¶¬¬¬¬XD_SOURCE_NAME¬=> D ( AS_SOURCE_NAME, PARENT¬)¶¬¬¬¬);¶¬       ENUM_LIT
     ERAL_LIST¬: SEQ_TYPE¬:= LIST (¬ENUM_LITERAL_S );¶¬       ENUM_LITERAL_FIRST¬: TREE¬:= HEAD (¬ENUM_LITERAL_LIST );¶¬       ENUM_LITERAL¬: TREE;¶¬
         ENUM_POS¬: INTEGER¬:= -1;¶¬     begin¶¬       D¬( LX_SRCPOS, RANGE_NODE, TREE_VOID );¶¬       ¶¬       D¬( SM_TYPE_SPEC, D (¬AS_SOURCE_NAME, PAREN
0060 T), ENUMERATION );¶¬       D¬( SM_BASE_TYPE, ENUMERATION, ENUMERATION );¶¬     ¶¬       while not IS_EMPTY ( ENUM_LITERAL_LIST) loop¶¬¬ POP ( ENUM_L
     ITERAL_LIST, ENUM_LITERAL );¶¬¬ WALK ( ENUM_LITERAL, ENUM_LITERAL_S, REGION );¶¬¬ D ( SM_OBJ_TYPE, ENUM_LITERAL, ENUMERATION );¶¬¬ ENUM_POS¬:= ENUM_PO
     S + 1;¶¬¬ DI ( SM_POS, ENUM_LITERAL, ENUM_POS );¶¬¬ DI ( SM_REP, ENUM_LITERAL, ENUM_POS );¶¬       end loop;¶¬     ¶¬       D¬( AS_EXP1, RANGE_NODE,
      MAKE_USED_OBJECT_ID (¶¬¬¬LX_SYMREP¬=> D ( LX_SYMREP, ENUM_LITERAL_FIRST ),¶¬¬ SM_EXP_TYPE¬=> ENUMERATION,¶¬¬ SM_VALUE¬=> UARITH.U_VAL ( 0¬),¶¬¬ SM_DE
     FN¬=> ENUM_LITERAL_FIRST¶¬¬ )¶¬¬ );¶¬       D¬( AS_EXP2, RANGE_NODE, MAKE_USED_OBJECT_ID (¶¬¬¬LX_SYMREP¬=> D ( LX_SYMREP, ENUM_LITERAL ),¶¬¬ SM_EXP_T
     YPE¬=> ENUMERATION,¶¬¬ SM_VALUE¬=> UARITH.U_VAL ( ENUM_POS ),¶¬¬ SM_DEFN¬=> ENUM_LITERAL¶¬¬ )¶¬¬ );¶¬       D¬( SM_TYPE_SPEC, RANGE_NODE, ENUMERATION
     ¬);¶¬     end;¶       ¶  when DN_SUBTYPE_INDICATION =>¶¬     declare¶¬       CONSTRAINT¬: constant TREE¬:= D ( AS_CONSTRAINT, NODE );¶¬       NAM
     E¬: constant TREE¬:= D ( AS_NAME, NODE );¶¬     begin¶¬       WALK ( CONSTRAINT, NODE, REGION¬);¶¬       WALK ( NAME, NODE, REGION );¶¬       D¬( A
     S_NAME, NODE, MAKE_USED_NAME_ID (¶¬¬¬SM_DEFN¬=> D ( SM_DEFN, NAME ),¶¬¬ LX_SYMREP¬=> D ( LX_SYMREP, NAME ),¶¬¬ LX_SRCPOS¬=> D ( LX_SRCPOS, NAME )¶¬¬ )
     ¶¬¬ );¶¬     end;¶       ¶  when DN_INTEGER_DEF =>¶¬     declare¶¬       use UARITH;¶¬       CONSTRAINT¬: constant TREE¬:= D ( AS_CONSTRAINT, NOD
0070 E );¶¬       INTEGER_NODE¬: TREE¬:= MAKE_INTEGER (¶¬¬¬¬SM_RANGE¬=> CONSTRAINT,¶¬¬¬XD_SOURCE_NAME¬=> D ( AS_SOURCE_NAME, PARENT¬)¶¬¬¬);¶¬     begin¶¬
          WALK ( CONSTRAINT, NODE, REGION¬);¶¬       D¬( SM_TYPE_SPEC, CONSTRAINT, INTEGER_NODE );¶¬       D¬( SM_TYPE_SPEC, D (¬AS_SOURCE_NAME, PARENT
     ), INTEGER_NODE);¶¬       D¬( SM_BASE_TYPE, INTEGER_NODE,¬INTEGER_NODE);¶¬     end;¶       ¶¬  when DN_FLOAT_DEF¬=>¶¬     declare¶¬       use UAR
     ITH;¶¬     ¶¬       CONSTRAINT¬: constant TREE¬:= D ( AS_CONSTRAINT, NODE );¶¬       RANGE_NODE¬: TREE¬:= D ( AS_RANGE, CONSTRAINT );¶¬       FLOAT
     _NODE¬: TREE¬:= MAKE_FLOAT (¶¬¬¬¬SM_RANGE¬=> RANGE_NODE,¶¬¬¬XD_SOURCE_NAME¬=> D ( AS_SOURCE_NAME, PARENT¬)¶¬¬¬);¶¬     begin¶¬       WALK ( CONSTRAIN
     T, NODE, REGION¬);¶¬       D¬( SM_TYPE_SPEC, CONSTRAINT, FLOAT_NODE );¶¬       D¬( SM_TYPE_SPEC, RANGE_NODE, FLOAT_NODE );¶¬       D¬( SM_ACCURACY,
      FLOAT_NODE, D ( SM_VALUE, D ( AS_EXP, CONSTRAINT ) ) );¶¬       D¬( SM_TYPE_SPEC, D (¬AS_SOURCE_NAME, PARENT ), FLOAT_NODE );¶¬       D¬( SM_BASE_T
     YPE, FLOAT_NODE, FLOAT_NODE );¶¬     end;¶       ¶  when DN_FIXED_DEF¬=>¶¬     declare¶¬       use UARITH;¶¬       CONSTRAINT¬: constant TREE¬:=
     D ( AS_CONSTRAINT, NODE );¶¬       RANGE_NODE¬: TREE¬:= D ( AS_RANGE, CONSTRAINT );¶¬       FIXED_NODE¬: TREE¬:= MAKE_FIXED ( SM_RANGE => RANGE_NODE
     ,¬XD_SOURCE_NAME => D¬( AS_SOURCE_NAME, PARENT) );¶¬     begin¶¬       WALK ( CONSTRAINT, NODE, REGION);¶¬       D¬( SM_TYPE_SPEC, CONSTRAINT, FIXED
0080 _NODE );¶¬       D¬( SM_TYPE_SPEC, RANGE_NODE, FIXED_NODE );¶¬       D¬( SM_ACCURACY, FIXED_NODE, D ( SM_VALUE, D ( AS_EXP, CONSTRAINT ) ) );¶¬
        D¬( SM_TYPE_SPEC, D (¬AS_SOURCE_NAME, PARENT ), FIXED_NODE );¶¬       D¬( SM_BASE_TYPE, FIXED_NODE, FIXED_NODE );¶¬       D¬( CD_IMPL_SMALL, FIXE
     D_NODE, D ( SM_ACCURACY, FIXED_NODE ) );¶¬     end;¶       ¶  when DN_UNCONSTRAINED_ARRAY_DEF =>¶¬     declare¶¬       SUBTYPE_INDICATION¬: consta
     nt TREE¬:= D ( AS_SUBTYPE_INDICATION,¬NODE );¶¬       INDEX_S¬: constant TREE¬:= D ( AS_INDEX_S, NODE );¶¬     ¶¬       ARRAY_NODE:¬TREE := MAKE_ARR
     AY (¬SM_INDEX_S¬=> INDEX_S,¶¬¬¬¬SM_SIZE¬=> TREE_VOID,¶¬¬¬XD_SOURCE_NAME¬=> D ( AS_SOURCE_NAME, PARENT¬)¶¬¬¬);¶¬     begin¶¬       WALK ( SUBTYPE_INDIC
     ATION, NODE, REGION );¶¬       WALK ( INDEX_S, NODE,¬REGION );¶¬       D¬( SM_COMP_TYPE, ARRAY_NODE, TYPE_SPEC_FOR_SUBTYPE¬( SUBTYPE_INDICATION ) );
     ¶¬       D¬( SM_TYPE_SPEC, D (¬AS_SOURCE_NAME, PARENT ), ARRAY_NODE );¶¬       D¬( SM_BASE_TYPE, ARRAY_NODE, ARRAY_NODE );¶¬     end;¶       ¶¬  w
     hen DN_USED_CHAR¬=>¶¬     declare¶¬       DEFN: TREE := HEAD_DEFN ( NODE);¶¬     begin¶¬       D¬( SM_DEFN, NODE, DEFN);¶¬       D¬( SM_EXP_TYPE, N
     ODE, D ( ¶¬¬¬¬¬¬       SM_OBJ_TYPE,DEFN));¶¬       D¬( SM_VALUE, NODE, UARITH.U_VAL(DI ( ¶¬¬¬¬¬¬       SM_POS,¶¬¬¬¬¬¬       DEFN)));¶¬     end;¶       ¶¬
     when DN_USED_OBJECT_ID =>¶¬     declare¶¬       DEFN¬: TREE¬:= HEAD_DEFN ( NODE¬);¶¬     begin¶¬       D¬( SM_DEFN, NODE, DEFN );¶¬       if DEFN.T
0090 Y = DN_ENUMERATION_ID then¶¬¬ D ( SM_EXP_TYPE, NODE, D ( SM_OBJ_TYPE, DEFN ) );¶¬¬ D ( SM_VALUE, NODE, UARITH.U_VAL ( DI ( SM_POS, DEFN ) ) );¶¬
       end if;¶¬     end;¶       ¶¬  when DN_FUNCTION_CALL =>¶¬     declare¶¬       use UARITH;¶¬       use PRENAME;¶¬     ¶¬       NAME¬: constant TR
     EE¬:= D ( AS_NAME, NODE );¶¬       GENERAL_ASSOC_S¬: constant TREE¬:= D ( AS_GENERAL_ASSOC_S, NODE );¶¬       PARAM¬: TREE¬:= HEAD (¬LIST ( GENERAL_
     ASSOC_S ) );¶¬       PARAM2¬: TREE¬:= TREE_VOID;¶¬       BLTN_OPERATOR_ID¬: TREE¬:= HEAD (¬LIST ( D ( LX_SYMREP, NAME ) ) );¶¬     begin¶¬¬¬¬¬-- ONLY
      FOR UNARY "-", "*", "**" IN RANGES¶¬       WALK ( GENERAL_ASSOC_S, NODE, REGION );¶¬     ¶¬       if not IS_EMPTY ( TAIL ( LIST (¬GENERAL_ASSOC_S))
     ) then¶¬¬ PARAM2 := HEAD ( TAIL ( LIST¬( GENERAL_ASSOC_S )¬) );¶¬       end if;¶¬     ¶¬       if (PARAM2 = TREE_VOID) xor ( OP_CLASS'VAL ( DI ( SM_
     OPERATOR, BLTN_OPERATOR_ID )¬) in CLASS_UNARY_OP¬) then¶¬¬ BLTN_OPERATOR_ID := HEAD ( TAIL ( LIST¬( D ( LX_SYMREP, NAME ) ) ) );¶¬       end if;¶¬
       ¶¬     D¬( AS_NAME, NODE, MAKE_USED_OP¬(¶¬¬¬SM_DEFN¬=> BLTN_OPERATOR_ID,¶¬¬¬LX_SYMREP¬=> D ( LX_SYMREP, NAME), LX_SRCPOS => D¬( LX_SRCPOS, NAME )
     ¶¬¬¬)¶¬¬ );¶¬     ¶¬       if PRINT_NAME ( D ( LX_SYMREP, NAME ) ) =¬"""-""" then¶¬¬ if PARAM2 = TREE_VOID then¶¬¬     D ( SM_VALUE, NODE, -D( SM_VALU
     E, PARAM ) );¶¬¬ else¶¬¬     D ( SM_VALUE, NODE, D ( SM_VALUE, PARAM ) - D¬( SM_VALUE, PARAM2 ) );¶¬¬ end if;¶¬       elsif PRINT_NAME ( D ( LX_SYMREP

0100 , NAME ) ) = """*""" then¶¬¬ D ( SM_VALUE, NODE, D ( SM_VALUE, PARAM ) * D ( SM_VALUE, PARAM2 ) );¶¶¬       elsif PRINT_NAME ( D ( LX_SYMREP, NAME )
     ) = """**""" then¶¬¬ D ( SM_VALUE, NODE, D ( SM_VALUE, PARAM ) ** D (¬SM_VALUE,¬PARAM2 ) );¶¶¬       else¶¬¬ ABORT_RUN ( "FUNCTION NOT ALLOWED - " &
     PRINT_NAME ( D ( LX_SYMREP, NAME ) ) );¶¬¬ raise PROGRAM_ERROR;¶¬       end if;¶¬     ¶¬       D¬( SM_EXP_TYPE, NODE, GET_BASE_TYPE ( D ( SM_EXP_TYP
     E, PARAM¬) ) );¶¬       D¬( SM_NORMALIZED_PARAM_S, NODE, MAKE_EXP_S (¶¬¬¬LIST¬=> LIST (¬GENERAL_ASSOC_S ),¶¬¬¬LX_SRCPOS¬=> D ( LX_SRCPOS, GENERAL_ASS
     OC_S)¶¬¬¬)¶¬¬ );¶¶¶     end;¶       ¶  when DN_NUMERIC_LITERAL =>¶¬     declare¶¬       VALUE¬: TREE¬:= UARITH.U_VALUE( PRINT_NAME( D( LX_NUMREP,
      NODE¬) ) );¶¬     begin¶¬       if (VALUE.PT = HI and¬then VALUE.NOTY = DN_NUM_VAL)¶¬¬ or (VALUE.PT = P and then VALUE.TY = DN_NUM_VAL)¶¬       the
     n¶¬¬ D( SM_EXP_TYPE, NODE, MAKE(¬DN_UNIVERSAL_INTEGER ) );¶¬       else¶¬¬ D( SM_EXP_TYPE, NODE, MAKE(¬DN_UNIVERSAL_REAL )¬);¶¬       end if;¶¬
        D( SM_VALUE, NODE, VALUE );¶¬     end;¶       ¶  when DN_RANGE =>¶¬     declare¶¬       EXP1¬: constant TREE := D( AS_EXP1, NODE );¶¬       EX
     P2¬: constant TREE := D( AS_EXP2, NODE );¶¬     begin¶¬       WALK( EXP1,¬NODE, REGION );¶¬       WALK( EXP2,¬NODE, REGION );¶¬       D( SM_TYPE_SP
     EC, NODE, GET_BASE_TYPE( D( SM_EXP_TYPE, EXP1 ) ) );¶¬     end;¶       ¶  when DN_DISCRETE_SUBTYPE =>¶¬     declare¶¬       SUBTYPE_INDICATION¬: c
0110 onstant TREE := D( AS_SUBTYPE_INDICATION, NODE¬);¶¬     begin¶¬       WALK( SUBTYPE_INDICATION, NODE,¬REGION );¶¬     end;¶       ¶  when DN_FLOAT
     _CONSTRAINT =>¶¬     declare¶¬       EXP¬: constant TREE := D( AS_EXP,¬NODE );¶¬       RANGE_NODE¬: constant TREE := D( AS_RANGE, NODE );¶¬     begi
     n¶¬       WALK( EXP, NODE, REGION );¶¬       WALK( RANGE_NODE, NODE, REGION );¶¬     end;¶       ¶  when DN_FIXED_CONSTRAINT =>¶¬     declare¶¬
          EXP¬: constant TREE := D( AS_EXP,¬NODE );¶¬       RANGE_NODE¬: constant TREE := D( AS_RANGE, NODE );¶¬     begin¶¬       WALK( EXP, NODE, REGI
     ON );¶¬       WALK( RANGE_NODE, NODE, REGION );¶¬     end;¶       ¶  when DN_PACKAGE_SPEC =>¶¬     declare¶¬       DECL_S1 : constant TREE := D (
      AS_DECL_S1, NODE );¶¬       DECL_S2 : constant TREE := D ( AS_DECL_S2, NODE );¶¬     begin¶¬       WALK ( DECL_S1, NODE,¬REGION );¶¬       WALK (
     DECL_S2, NODE,¬REGION );¶¬       LIST ( D ( AS_DECL_S2, NODE), (TREE_NIL,TREE_NIL) );¬¬--| PAS DE PARTIE PRIVEE (REP¬SPECS SEULEMENT)¶¬     end;¶
          ¶¬  when DN_COMPILATION =>¶¬     declare¶¬       COMPLTN_UNIT_S : constant TREE := D ( AS_COMPLTN_UNIT_S, NODE¬);¶¬     begin¶¬       WALK ( CO
     MPLTN_UNIT_S, NODE, REGION );¶¬     end;¶       ¶¬  when DN_COMPILATION_UNIT =>¶¬     declare¶¬       CONTEXT_ELEM_S¬: constant TREE := D ( AS_CONT
     EXT_ELEM_S, NODE );¶¬       ALL_DECL¬: constant TREE := D ( AS_ALL_DECL, NODE );¶¬       PRAGMA_S¬: constant TREE := D ( AS_PRAGMA_S, NODE );¶¬
0120 begin¶¬       WALK ( CONTEXT_ELEM_S, NODE, REGION );¶¬       WALK ( ALL_DECL, NODE, REGION );¶¬       WALK ( PRAGMA_S, NODE, REGION );¶¬       DI
     ( XD_TIMESTAMP, NODE, 1 );¶¬       LIST ( NODE, (TREE_NIL,TREE_NIL) );¶¬       D¬( XD_LIB_NAME, NODE, STORE_SYM ( "_STANDRD.DCL" )¬);¶¬     end;¶
```

```
          ¶¬  when DN_INDEX =>¶¬     declare¶¬        NAME¬: constant TREE := D ( AS_NAME, NODE );¶¬     begin¶¬        WALK ( NAME, NODE, REGION );¶¬
      D¬( SM_TYPE_SPEC, NODE, D ( SM_TYPE_SPEC,¬D (SM_DEFN, NAME ) ) );¶¬        D¬( AS_NAME, NODE, MAKE_USED_NAME_ID (¶¬¬¬LX_SRCPOS¬=> D ( LX_SRCPOS, N
AME ),¶¬¬¬LX_SYMREP¬=> D ( LX_SYMREP, NAME ),¶¬¬¬SM_DEFN¬=> D ( SM_DEFN, NAME )¶¬¬¬)¶¬¬ );¶¬    end;¶        ¶¬  when others =>¶¬     declare¶¬
   ITEM_LIST¬: SEQ_TYPE;¶¬        ITEM_NODE¬: TREE;¶¬     begin¶¬        case ARITY ( NODE ) is¶¬¬ when NULLARY =>¶¬¬    null;¶¬¬ when UNARY =>¶¬¬
WALK ( SON_1 ( NODE), NODE, REGION );¶¬¬ when BINARY =>¶¬¬    WALK ( SON_1 ( NODE), NODE, REGION );¶¬¬    WALK ( SON_2 ( NODE), NODE, REGION );¶¬¬ whe
n TERNARY =>¶¬¬    WALK ( SON_1 ( NODE), NODE, REGION );¶¬¬    WALK ( SON_2 ( NODE), NODE, REGION );¶¬¬    WALK ( SON_3 ( NODE), NODE, REGION );¶¬¬ wh
en ARBITRARY =>¶¬¬    ITEM_LIST := LIST ( NODE);¶¬¬    while¬not IS_EMPTY ( ITEM_LIST ) loop¶¬¬       POP ( ITEM_LIST, ITEM_NODE );¶¬¬       WALK (  I
TEM_NODE, NODE, REGION );¶¬¬    end loop;¶¬        end case;¶¬     end;¶        ¶        end case;¶¶      end¬WALK;¶      --|------------------------
0130 ------------------------------------------------------------------¶      --|¬PROCEDURE¬MAKE_PREDEF_IDS¶       procedure MAKE_PREDEF_IDS ( ID_LIST :o
ut SEQ_TYPE ) is¶         use PRENAME;¶      ¶         NEW_ID_LIST¬: SEQ_TYPE¬:= (TREE_NIL, TREE_NIL);¶        NEW_ID¬¬: TREE;¶        NEW_ARG_LIST¬
: SEQ_TYPE;¶         NEW_ARG¬¬: TREE;¶        ITEM_LENGTH¬: NATURAL;¶         use MAKE_NOD;¶      begin¶        for PRAGMA_NAME in DEFINED_PRAGMAS l
oop¶¬  NEW_ARG_LIST := (TREE_NIL, TREE_NIL);¶¬  if PRAGMA_NAME = LIST or PRAGMA_NAME = PRENAME.DEBUG then¶¬     for ARG_NAME in LIST_ARGUMENTS loop¶¬
       NEW_ARG := MAKE_ARGUMENT_ID (¶¬¬¬LX_SYMREP¬=> STORE_SYM ( LIST_ARGUMENTS'IMAGE ( ARG_NAME ) ),¶¬¬¬XD_POS¬=> LIST_ARGUMENTS'POS ( ARG_NAME )¶¬¬¬
);¶¬        NEW_ARG_LIST := APPEND ( NEW_ARG_LIST, NEW_ARG );¶¬     end loop;¶¬  elsif PRAGMA_NAME¬= OPTIMIZE then¶¬     for ARG_NAME in OPTIMIZE_ARGU
MENTS¬loop¶¬        NEW_ARG := MAKE_ARGUMENT_ID (¶¬¬¬LX_SYMREP¬=> STORE_SYM ( OPTIMIZE_ARGUMENTS'IMAGE¬( ARG_NAME ) ),¶¬¬¬XD_POS¬=> OPTIMIZE_ARGUMENTS
'POS ( ARG_NAME )¶¬¬¬);¶¬        NEW_ARG_LIST := APPEND ( NEW_ARG_LIST, NEW_ARG );¶¬     end loop;¶¬  elsif PRAGMA_NAME¬= SUPPRESS then¶¬     for ARG_
NAME in SUPPRESS_ARGUMENTS¬loop¶¬        NEW_ARG := MAKE_ARGUMENT_ID (¶¬¬¬LX_SYMREP¬=> STORE_SYM ( SUPPRESS_ARGUMENTS'IMAGE¬( ARG_NAME ) ),¶¬¬¬XD_POS¬
=> SUPPRESS_ARGUMENTS'POS ( ARG_NAME )¶¬¬¬);¶¬        NEW_ARG_LIST := APPEND ( NEW_ARG_LIST, NEW_ARG );¶¬     end loop;¶¬  elsif PRAGMA_NAME¬= INTERFA
0140 CE then¶¬     for ARG_NAME in INTERFACE_ARGUMENTS loop¶¬        NEW_ARG := MAKE_ARGUMENT_ID (¶¬¬¬LX_SYMREP¬=> STORE_SYM ( INTERFACE_ARGUMENTS'IMAGE (
ARG_NAME ) ),¶¬¬¬XD_POS¬=> INTERFACE_ARGUMENTS'POS ( ARG_NAME )¶¬¬¬);¶¬        NEW_ARG_LIST := APPEND ( NEW_ARG_LIST, NEW_ARG );¶¬     end loop;¶¬  en
d if;¶¬  declare¶¬     SYM¬: TREE¬:= STORE_SYM ( DEFINED_PRAGMAS'IMAGE ( PRAGMA_NAME ) );¶¬  begin¶¬     NEW_ID := MAKE_PRAGMA_ID¬(¶¬¬LX_SYMREP¬=> SYM
,¶¬¬XD_POS¬=> DEFINED_PRAGMAS'POS ( PRAGMA_NAME ),¶¬¬SM_ARGUMENT_ID_S¬=> MAKE_ARGUMENT_ID_S ( LIST => NEW_ARG_LIST )¶¬¬);¶¬     NEW_ID_LIST :=¬APPEND
( NEW_ID_LIST, NEW_ID );¶¬     LIST¬( SYM, INSERT ( LIST ( SYM ),¬NEW_ID ) );¶¬  end;¶        end loop;¶        ¶        for ATTRIBUTE_NAME in DEFINED
_ATTRIBUTES¬loop¶¬  declare¶¬     ITEM_NAME¬: constant STRING¬:= DEFINED_ATTRIBUTES'IMAGE (¬ATTRIBUTE_NAME );¶¬     SYM¬¬: TREE;¶¬  begin¶¬     ITEM_L
ENGTH :=¬ITEM_NAME'LENGTH;¶¬     if ITEM_NAME( ITEM_LENGTH - 1 ..ITEM_LENGTH ) = "_X" then¶¬        ITEM_LENGTH¬:= ITEM_LENGTH - 2;¶¬     end if;¶¬
  SYM := STORE_SYM (ITEM_NAME( 1..ITEM_LENGTH ) );¶¬     NEW_ID := MAKE_ATTRIBUTE_ID (¶¬¬¬LX_SYMREP¬=> SYM,¶¬¬XD_POS¬=> DEFINED_ATTRIBUTES'POS ( ATTR
IBUTE_NAME )¶¬¬¬);¶¬     NEW_ID_LIST :=¬APPEND ( NEW_ID_LIST, NEW_ID );¬¬¬¬--| PREFIXER  LA LISTE DES IDS¶¬     LIST¬( SYM, INSERT ( LIST ( SYM ),¬NEW
_ID ) );¬¬¬¬--| CHANGER LA XD_DEFLIST PAR¬UNE AUGMENTEE EN FIN DE L'ID CRE¶¬  end;¶        end loop;¶      ¶         for OP_NAME in OP_CLASS loop¶¬

0150 declare¶¬     ITEM_NAME¬: constant STRING¬:= BLTN_TEXT_ARRAY ( OP_NAME );¶¬     SYM¬¬: TREE;¶¬  begin¶¬     ITEM_LENGTH :=¬3;¶¬     while ITEM_NAME( I
TEM_LENGTH ) = '!' loop¶¬        ITEM_LENGTH¬:= ITEM_LENGTH - 1;¶¬     end loop;¶¬     SYM := STORE_SYM ( '"' &¬ITEM_NAME( 1..ITEM_LENGTH ) &¬'"' );¶¬
     NEW_ID := MAKE_BLTN_OPERATOR_ID (¶¬¬¬LX_SYMREP¬=> SYM,¶¬¬¬SM_OPERATOR¬=> OP_CLASS'POS ( OP_NAME )¶¬¬¬);¶¬     NEW_ID_LIST :=¬APPEND ( NEW_ID_LIST
, NEW_ID );¬¬¬¬¬--| PREFIXER  LA LISTE DES IDS¶¬     LIST¬( SYM, INSERT ( LIST ( SYM ),¬NEW_ID ) );¬¬¬¬¬--| CHANGER LA XD_DEFLIST PAR¬UNE AUGMENTEE EN F
IN DE L'ID CRE¶¬  end;¶        end loop;¶      ¶         ID_LIST :=¬NEW_ID_LIST;¬¬¬¬¬¬¬¬--| RENDRE LA LISTE¬DES IDS¶        ¶      end¬MAKE_PREDEF_ID
S;¶   ¶   ¶begin¶  declare¶    USER_ROOT¬¬: TREE;¶    PREDEF_ID_LIST¬¬: SEQ_TYPE;¶  begin¶    USER_ROOT := D(¬XD_USER_ROOT, TREE_ROOT );¶    MAKE_PRED
EF_IDS( PREDEF_ID_LIST );¬¬¬¬¬¬¬¬--| NOEUDS STANDARD¬POUR LES NOMS PREDEFINIS¶      ¶    WALK(¬D( XD_STRUCTURE, USER_ROOT ),¬PARENT =>¬TREE_VOID, REGIO
N => TREE_VOID );¬¬¬--| PARCOURIR L'ARBRE SYNTAXIQUE DU _STANDRD¶      ¶    declare¶      INTEGER_ID¬¬: TREE¬:= HEAD_DEFN( STORE_SYM( "INTEGER" ) );¶
     NATURAL_ID¬¬: TREE¬:= HEAD_DEFN( STORE_SYM( "NATURAL" ) );¶      POSITIVE_ID¬¬: TREE¬:= HEAD_DEFN( STORE_SYM( "POSITIVE" ) );¶      INTGR_SIZE¬¬:
INTEGER¬:= DI( CD_IMPL_SIZE, D( SM_TYPE_SPEC, INTEGER_ID ) );¶      DURATION_ID¬¬: TREE¬:= HEAD_DEFN( STORE_SYM( "DURATION" ) );¶      DURATION_BASE_I
0160 D¬: TREE¬:= HEAD_DEFN( STORE_SYM( "_DURATION" ) );¶      DURATION_SPEC¬¬: TREE¬:= D( SM_TYPE_SPEC,¬DURATION_ID );¶      DURATION_BASE_SPEC¬: TREE¬:= D
( SM_TYPE_SPEC,¬DURATION_BASE_ID );¶    begin¶      DI(¬CD_IMPL_SIZE, D( SM_TYPE_SPEC,NATURAL_ID ), INTGR_SIZE );¶      DI(¬CD_IMPL_SIZE, D( SM_TYPE_S
PEC,POSITIVE_ID ), INTGR_SIZE);¶      DI(¬CD_IMPL_SIZE, DURATION_BASE_SPEC, DI( CD_IMPL_SIZE, DURATION_SPEC ) );¶      D (¬SM_BASE_TYPE, DURATION_SPEC
, DURATION_BASE_SPEC );¶      DB(¬SM_IS_ANONYMOUS, DURATION_BASE_SPEC, TRUE );¶      D (¬XD_SOURCE_NAME, DURATION_BASE_SPEC, DURATION_ID );¶      D (¬
SM_TYPE_SPEC, D( SM_RANGE, DURATION_SPEC ), DURATION_BASE_SPEC );¬¬¬--| SOUS TYPE CONTRAINTE D'ETENDUE POUR¬DURATION¶    end;¶      ¶    declare¶
 ADDRESS_ID¬: TREE¬:= HEAD_DEFN( STORE_SYM( "_ADDRESS" ) );¶      BASE_SPEC¬: TREE¬:= D( SM_TYPE_SPEC,¬D( SM_TYPE_SPEC, ADDRESS_ID )¬);¬¬--| ID DE¬TYP
E ANCTRE DANS SM_TYPE_SPEC¶      NEW_SPEC¬: TREE¬:= COPY_NODE( BASE_SPEC );¶    begin¶      D( XD_SOURCE_NAME, NEW_SPEC, ADDRESS_ID );¶      D( SM_DER
IVED, NEW_SPEC, BASE_SPEC );¶      D( SM_BASE_TYPE, NEW_SPEC, NEW_SPEC );¶      D( SM_TYPE_SPEC, ADDRESS_ID, NEW_SPEC );¶    end;¶      ¶    declare¶
     PACK_SYM¬: TREE¬:= HEAD_DEFN( STORE_SYM( "_STANDRD" ) );¬¬¬--| CHERCHER LE SYMBOLE NOM DU PACKAGE _STANDRD EN VERIFIANT QU'IL A UNE DEFLIST¶
 HEADER¬¬: TREE :=¬D( SM_SPEC, PACK_SYM );¶    begin¶      LIST( D( AS_DECL_S2, HEADER ), PREDEF_ID_LIST );¬¬¬¬¬¬--| IDENTIFICATEURS¬EN PARTIE¬PRIVEE¶
0170    end;¶         ¶  end;¶--|-----------------------------------------------------------------------------------------¶end FIX_PRE;¶
```